

Freight Rate Prediction

Machine Learning Engineer Assessment

Yash Yennam

Training data covers January–October 2025 (48,000 loads); predictions are required for November–December 2025 (12,000 loads), plus a fixed single-lane scenario for every day of December. The final model reaches **mean MAE \$106.18 / MAPE 4.60% / R² 0.8302** under rolling-origin validation, against \$129.78 for a plain gradient-boosted model and \$187.36 for a lane-mean baseline. Those aggregates are dominated by a thin tail of anomalous loads; on typical freight the median absolute percentage error is 1.40%.

Code: github.com/yashyennam/Freight_Rate_ML_Assessment

1. Data split and validation approach

The split is rolling-origin and strictly forward in time: train on every load up to and including month m , predict month $m+1$, for m = April through September. Six folds, each a miniature of the real task — fit on the past, predict a month that has not happened yet.

A random split was rejected **on evidence, not principle**. It overstates accuracy — \$90.94 MAE against \$106.18 for the same booster scored forward in time — and, far more damagingly, selects the wrong feature set. The reason is a single column.

1.1 Why a random split fails here: quote_signal

quote_signal is a rate-per-mile quote. For about half of all training rows it equals the realised rate-per-mile to within $\pm 2\%$, which makes it look like the single most valuable feature in the dataset. It is not stable.

Regressing it against load characteristics month by month reveals three distinct regimes:

Months	corr(quote_signal, log distance)	Equipment ordering	Regime
Jan, Feb, Mar, Jun, Sep	−0.80	Reefer > Flatbed > Dry Van ✓	Genuine quote
Apr, May, Jul, Oct	+0.81 (sign inverted)	Dry Van > Flatbed > Reefer ✗	Corrupted
Aug, Nov, Dec	≈ 0.00	all equal at ~2.05	Pure noise

The underlying economics are unambiguous: shorter hauls cost more per mile, and reefer costs more than flatbed, which costs more than dry van. Months where the sign flips are inverted copies; the months where the structure vanishes entirely carry no information at all.

November and December — every row requiring a prediction — are in the third regime. Within August, the one labelled month that shares it, the correlation between quote_signal and the actual rate per mile is −0.014. The column is dropped from the final feature set. The cost of the alternative is measurable:

Split	With quote_signal	Without	Verdict
Random 80/20	MAE \$90.94	\$97.96	says keep it
Forward → October	MAE \$126.84	\$159.10	says keep it
Forward → August (matches Nov/Dec regime)	\$149.40	MAE \$111.10	says drop it

August is treated as the dress-rehearsal fold throughout, because it is the only labelled month whose quote_signal regime matches the prediction window. There, the feature that a random split would have selected inflates error by 34%.

1.2 Rolling-origin results

Train through	Test month	Rows	Plain LightGBM	Hybrid (final)
April	May	4,913	\$96.26	\$86.16
May	June	4,783	\$138.88	\$118.98
June	July	4,912	\$102.27	\$120.23
July	August	4,759	\$111.10	\$104.66
August	September	4,670	\$171.08	\$107.57
September	October	4,853	\$159.10	\$99.50
Mean MAE			\$129.78	\$106.18

The hybrid wins five of six folds. July is the exception: June was the market peak and the level fell afterwards, so any model projecting forward from it overshoots.

Two rules keep the folds honest. The outlier filter is a training decision only, so each fold's test half keeps every extreme load — filtering globally before splitting would have removed ~1% of each test month and cut the reported MAE roughly in half. And target encodings are computed inside each fold's training window, so a January training row cannot carry lane statistics drawn from the month it is being asked to predict.

1.3 Where the remaining error sits

On the October holdout the model is tight for almost every load — **median APE 1.40%** — but the mean is dragged by a small set of loads priced far from anything their features imply.

October fold	Rows	MAE	MAPE	median APE
All rows	4,853	\$99.50	4.69%	1.40%
Excluding \$/mi outliers	4,793	\$50.80	2.24%	1.40%

Sixty rows — 1.2% of the fold — account for the entire gap. The **0.42% of rows with APE above 25% carry 95.8% of the total squared error**, which is why RMSE sits an order of magnitude above MAE. They are not identifiable in advance, and no adjustment helps: the conditional mean is already the right prediction for them.

Accuracy is even across segments — median APE is 1.37% (Dry Van), 1.37% (Reefer), 1.46% (Flatbed), and 1.31–1.48% across every distance band — so there is no segment the model quietly fails on. Calibration drifts slightly negative on the most expensive loads (mean bias −\$16 in the top predicted-rate quintile, about −0.4%).

2. Data quality

Check	Count	Action
Rows read	48,000	—
Duplicate load_id / duplicate rows	0 / 0	—
Missing weight	300	median impute + missing flag
Missing market_index	374	same-day mean impute + missing flag
Non-positive posted_rate	0	—
\$/mi outside [0.75, 6.0]	513 (1.07%)	dropped from training only
Rows used for training	47,487	—

- market_index is a market-wide daily signal — between-day variance is 6.6× the within-day variance — so a same-day mean is a far better fill than a global constant. weight has no such structure and gets a median.
- Outlier bounds come from the empirical distribution: the 0.1% tails sit at 0.44 and 10.1 \$/mi against a median of 2.15. The filter earns its place: training with the outliers left in and scoring on the same untouched test folds gives \$120.30 MAE in October against \$99.50 filtered, and \$115.63 against \$107.57 in September, with RMSE unchanged either way.
- Eight cities appear only in the validation set (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego). The model never keys on city identity: geography enters as latitude/longitude, midpoint, great-circle distance, bearing and circuitry, and lane/city target encodings are prior-smoothed so unseen categories fall back to the global prior exactly.
- Coordinates were verified internally consistent — each of the 72 cities has exactly one lat/lon, identical whether it appears as origin or destination. Road distance runs ~1.18× great-circle distance, a realistic circuitry factor.

3. Model

3.1 The problem the model has to solve

Aggregating to daily level and controlling for load mix, the rate level decomposes as:

$$\text{daily level} = 0.143 \times \text{market_index} + 0.00021 \times t$$

$R^2 = 0.826$ on daily aggregates. Neither term suffices alone — market_index gives 0.478, time gives 0.261. The trend works out to **+0.64% per month**, and it is not a repackaging of the market index: September and October have a low market index (0.89, 0.96, comparable to January's 0.93) yet sit about 6% higher in rate.

This is the central modelling problem. Predictions are needed 30–90 days past the end of training, and **a tree cannot extrapolate a trend** — it flattens at the last bucket it ever saw. Left to itself, LightGBM used market_index to memorise individual day levels, which is why removing that feature entirely improved its forward score by \$27 MAE.

3.2 Architecture

A linear backbone that extrapolates, plus a gradient-boosted residual for cross-sectional structure:

```
log(rate / distance) = linear( market_index, t, log_distance, equipment )
                        + LightGBM( lane, geography, weight, calendar position, ... )
```

The split is deliberate. The backbone owns the two quantities that must project past the training window and whose relationship is smooth — the market index and the drift — so they keep extrapolating linearly instead of flattening. Everything else is cross-sectional and stationary, so LightGBM fits it on what the backbone leaves behind. `market_index` and the time index are **withheld** from the booster to stop it re-memorising day levels.

Backbone term	Coefficient	Reading
<code>market_index</code>	+0.14313	a 0.1 rise in the index adds ~1.4% to rate
trend (per 100 days)	+0.02122	+0.64% per month
<code>log_distance</code>	-0.12594	doubling haul length cuts \$/mi by ~8.4%
Reefer	+0.11901	+12.6% over dry van
Flatbed	+0.07799	+8.1% over dry van

Every coefficient carries the sign economics predicts, which is the main reason to prefer this structure over a black box: the extrapolated part of the prediction is legible and can be argued with.

3.3 Target transform

The model predicts log rate-per-mile rather than the dollar rate. Rate is close to multiplicative in distance (corr = 0.91), so dividing distance out leaves the model to learn the part that is genuinely uncertain instead of re-learning the length of the haul. Logs also make the loss proportional: a \$200 miss on a 3,000-mile load is not the same mistake as a \$200 miss on a 300-mile load. Predictions are converted back with Duan's smearing estimate (1.00106 here), which cuts residual bias by roughly a third at no cost to MAE.

3.4 How much of the trend to extrapolate

Projecting a +0.64%/month drift into a window with no labels is the riskiest assumption in the model, so the amount projected was swept over the same rolling-origin folds rather than assumed:

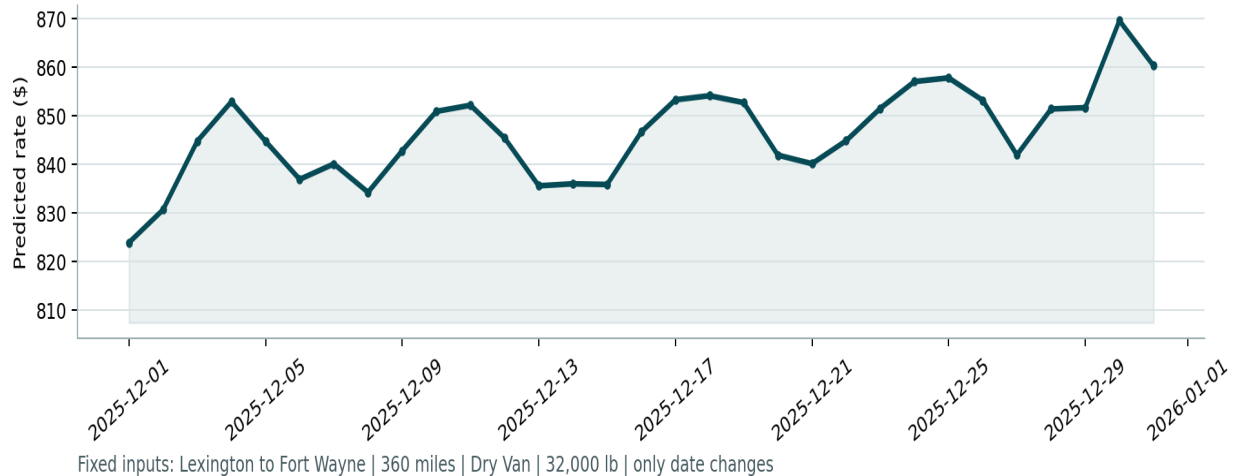
Trend damping	0.00	0.25	0.50	0.75	1.00
Mean MAE	\$110.19	\$110.64	\$111.19	\$111.83	\$112.54

Damping of 0 — hold the level at the last observed day — wins, and agrees with the raw series, where the level is flat across August–October (+4.3%, +6.1%, +6.3% against January). The spread is narrow and folds disagree in detail, so this is a mild preference rather than a strong result. It is also the conservative choice, which matters more at December's two-month horizon than at the one-month horizon the sweep could actually test.

4. Fixed December prediction chart

The December chart inputs ship with seven columns and **no `market_index`** — the strongest single driver of rate level. Every one of the 31 dates is present in `validation.csv`, and `market_index` is a market-wide daily quantity rather than a per-load one, so the daily mean across the ~200 validation loads on each December date is the correct value for the fixed scenario. It uses no information the model would not already have on those dates.

Candidate: December 2025 Predicted Load Rate



candidate_december.png — produced by the provided score.py

The curve for Lexington → Fort Wayne (360 mi, Dry Van, 32,000 lb) spans **\$823.85 – \$869.51**, a range of 5.5%. Two effects drive its shape:

- The daily market index, which produces the week-to-week undulation.
- An end-of-month ramp. After controlling for market level, equipment and distance, the day-of-month residual runs from -1.1% on the 1st to $+1.5\%$ on the 29th — a real month-end tightening worth about 2.5%, visible as the late-December rise. Day-of-week is negligible by comparison ($\sim 0.7\%$ total spread).

5. Honest limits

- The $+0.64\%$ /month drift is fitted over ten months. Ten months cannot separate trend from annual seasonality, so it is damped to zero beyond the training window rather than projected. If the true process is seasonal and December is a seasonal peak — which freight typically is — the December level is understated.
- The trend-damping sweep could only test a one-month horizon; December is two months out.
- `market_index` is taken as given for November and December. If it had to be forecast, the error bars would be materially wider.
- Roughly 0.4% of loads are priced far from anything their features imply. They are not predictable from the data provided, and they dominate RMSE.

6. Reproducing

```
python -m pip install -r requirements.txt
python -m src.predict          # writes both prediction files
python score.py --predictions validation_predictions.csv \
                --december-predictions data/december_chart_inputs.csv
python -m src.evaluate         # reproduces every validation experiment
```

RANDOM_STATE = 42 throughout; LightGBM is seeded and single-config, so the pipeline reproduces the submitted files byte-for-byte.